CropYield AI

Software Requirements Specification (SRS)

Machine-Learning-Based Crop Yield Prediction

Document version	1.0
Date	21 October 2026
Document status	Project baseline — subject to dataset validation
Intended audience	Student developer, project supervisor, reviewers, and portfolio evaluators

Purpose. This document defines the planned system's scope, interfaces, functional and non-functional requirements, data contract, ML evaluation rules, constraints, and acceptance criteria. It is designed to support implementation from a beginner-friendly baseline through a demonstrable portfolio application.

Project basis: a crop-yield prediction research paper by Hoque et al. (IEEE Access, 2024), combined with explicit engineering requirements and safeguards needed to implement and evaluate a reproducible ML application.

Document Control

Field	Description
Project	CropYield AI — crop yield prediction using machine learning
Document type	Software Requirements Specification (SRS)
Version	1.0
Status	Initial baseline; update after dataset inspection and supervisor review
Primary deliverable	Reproducible ML pipeline with a usable prediction interface and documented evaluation
Key dependency	Verification of the selected dataset's columns, units, licence, and target definition

Contents

- 1. Introduction
- 2. Overall Description
- 3. System Context and Architecture
- 4. Data Requirements
- 5. External Interface Requirements
- 6. Functional Requirements
- 7. Machine-Learning Requirements
- 8. Non-Functional Requirements
- 9. Use Cases and User Workflows
- 10. Acceptance Criteria and Verification
- 11. Project Phases and Deliverables
- 12. Risks, Assumptions, and Constraints
- 13. Deployment, Maintenance, and Future Enhancements
- 14. References and Glossary

1. Introduction

1.1 Purpose

This SRS specifies requirements for CropYield AI, a software system that analyses historical agricultural and environmental data and estimates crop yield for a selected crop and input conditions. It serves as the implementation baseline for data ingestion, exploratory analysis, model development, evaluation, inference, user interface, testing, and documentation.

1.2 Intended Audience

- Student developer: implement and test each module against the requirements.
- Project supervisor or evaluator: review scope, technical decisions, evidence, and completion status.
- End user or demonstrator: enter supported crop and environmental inputs and view an estimated yield with appropriate limitations.
- Future contributor: understand the data contract, model pipeline, architecture, and extension points.

1.3 Project Objective

Develop a reproducible, explainable, and usable machine-learning application that predicts crop yield from supported tabular inputs, compares multiple regression models, and reports performance using suitable regression metrics. The application is an educational and decision-support prototype; it is not a guarantee of actual harvest outcomes.

1.4 Scope

In scope

- Audit and document a historical crop-yield dataset and its licence/source.
- Load data from the selected supported format, initially CSV; XML/JSON adapters may be added where required by the selected source.
- Validate schema, types, missing values, duplicates, target units, and plausible ranges.
- Perform exploratory data analysis (EDA) and produce reproducible charts and summary tables.
- Train and compare a simple baseline and tree-based regression models, including Linear Regression and Gradient Boosting; optionally include KNN and Random Forest.
- Use a leakage-safe preprocessing and evaluation workflow, hyperparameter tuning, and a held-out test set.
- Save a versioned model pipeline and provide prediction through a simple Streamlit UI; a FastAPI service is an optional interface.
- Document limitations, model metrics, data provenance, setup steps, and results in a portfolio-ready repository.

Out of scope for the initial release

- Operational farm recommendations, guaranteed yield outcomes, or autonomous agronomic decisions.
- Live satellite ingestion, weather APIs, geospatial forecasting, and real-time sensor integration.
- Crop disease detection, fertilizer/pesticide prescriptions, and causal claims about input effects.
- Production-grade multi-tenant access control, payment, or large-scale cloud infrastructure.

1.5 Definitions and Acronyms

Term	Definition
AI / ML	Artificial Intelligence / Machine Learning.
EDA	Exploratory Data Analysis: profiling and visualizing data before modeling.
Feature (X)	An input variable used by a model, such as crop type or rainfall.
Target (y)	The output variable to predict: crop yield in a verified unit.
MAE	Mean Absolute Error; average absolute prediction error in target units.
RMSE	Root Mean Squared Error; error metric that penalizes larger errors more strongly.
R^2	Coefficient of determination; a comparative fit metric, not a percentage guarantee.
Data leakage	Use of information during training that would not legitimately be available at prediction time.
Pipeline	A packaged sequence of preprocessing and estimator steps fitted together.

2. Overall Description

2.1 Product Perspective

CropYield AI is a standalone Python application. Its core is a supervised regression pipeline trained on historical examples. A user-facing interface calls the same validated inference component used in testing, so predictions do not rely on a separate, inconsistent preprocessing implementation.

2.2 Product Functions

- Import and validate a dataset.
- Generate data-quality reports and EDA outputs.
- Create train/validation/test partitions and fit preprocessing only on training data.
- Train, compare, and tune candidate regression models.
- Report MAE, RMSE, and R^2, along with split strategy and target units.
- Persist the complete preprocessing-plus-model pipeline and metadata.
- Accept supported inputs, validate them, return a yield estimate, and display warnings for out-of-range or unsupported values.
- Provide documentation and tests that allow another developer to reproduce the workflow.

2.3 User Classes

User class	Needs	Access in initial version
Developer / maintainer	Run data preparation, training, tests, and model updates.	Local development environment and repository
Evaluator / reviewer	Inspect data decisions, metrics, limitations, and reproducibility.	Documentation, reports, and demo interface
Demo user	Enter supported values and view an estimated yield.	Streamlit interface; no account required

2.4 Operating Environment

- Operating system: Windows, macOS, or Linux; development may initially use Windows.
- Language/runtime: Python 3.10 or a compatible version selected and pinned during setup.
- Core libraries: pandas, NumPy, scikit-learn, Matplotlib, and Seaborn.

- Development: Jupyter Notebook or VS Code, Git, and a virtual environment.
- Optional serving layer: Streamlit for the user interface; FastAPI and Uvicorn if a separate API is needed.
- Model persistence: joblib or another approved serialization method for trusted local artifacts.

2.5 Design and Implementation Constraints

- The model is limited by the coverage, quality, and representativeness of the selected dataset.
- Feature names and units must be derived from the actual data dictionary and file; they must not be assumed from a paper alone.
- Training and inference must use the same fitted preprocessing pipeline.
- A result from a random split does not, by itself, prove performance on future years or unseen regions.
- The first release is intended as a portfolio prototype and must label estimates as estimates.

3. System Context and Architecture

The proposed design separates data handling, analysis, model training, inference, and presentation. Each component should have a clear responsibility and be testable independently.

Component	Responsibility	Expected output
Data source	Historical crop-yield and related environmental records.	Raw file plus source/licence metadata
Ingestion layer	Read supported file format; check file and schema.	DataFrame or clear validation error
Data validation / cleaning	Normalize column names/types; report missingness, duplicates, and invalid values.	Validated dataset and quality report
EDA and feature preparation	Summarize distributions, target, categories, correlations, and potential issues.	Charts, tables, documented decisions
Training pipeline	Split data, fit preprocessing and candidate regressors, tune within training data.	Fitted pipeline and experiment results
Evaluation / model report	Evaluate on held-out data and record metrics, split method, and limitations.	Metrics table and model card
Inference service	Validate input, load pipeline, transform inputs, predict, and return units.	Prediction plus status/warnings
User interface	Collect inputs and present estimate, units, and caveats.	Readable prediction screen

3.1 Proposed Data Flow

Historical dataset → Ingestion → Schema/data validation → EDA → Train/test split → Preprocessing pipeline → Model training/tuning → Held-out evaluation → Save approved pipeline → Input validation → Prediction UI/API → Prediction with unit and limitations.

3.2 Proposed Repository Structure

```
CropYield-AI/
■■■ app/                  # Streamlit UI or API entry point
■■■ data/
■   ■■■ raw/              # Original, unchanged dataset (not committed if restricted)
■   ■■■ interim/          # Intermediate cleaning outputs
■   ■■■ processed/        # Reproducible model-ready data
■■■ docs/                 # SRS, data dictionary, model card
■■■ models/               # Approved model pipeline and metadata
■■■ notebooks/            # EDA and experiments
■■■ reports/              # Figures, evaluation and data-quality reports
■■■ src/crop_yield_ai/
■   ■■■ data/             # Loaders and validation
■   ■■■ features/         # Feature/schema utilities
■   ■■■ training/         # Training, tuning, evaluation
■   ■■■ inference/        # Prediction and input validation
■■■ tests/                # Unit and integration tests
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
■■■ pyproject.toml (optional)
```

4. Data Requirements

4.1 Research Baseline

The motivating reference is Hoque et al., “Incorporating Meteorological Data and Pesticide Information to Forecast Crop Yields Using Machine Learning,” IEEE Access, 2024, DOI 10.1109/ACCESS.2024.3383309. The paper describes an India-focused historical dataset spanning 1990–2013 and six crops: rice, wheat, potato, soybean, sweet potato, and sorghum. It discusses yield, average rainfall, average temperature, and pesticide use.

> **Important:** the paper is a research baseline, not proof that the same exact file or every variable is available locally. Before coding the final schema, inspect the downloaded dataset, its source page, licence, column names, and units. Do not invent or silently rename missing variables.

4.2 Required Dataset Characteristics

- Source: documented dataset origin and version/date of retrieval.
- Scope: crop-yield records relevant to the supported geography, crops, and years.
- Target: one numeric yield column with a clearly verified unit.
- Predictors: only fields available at prediction time and present in the actual dataset.
- Rows: one clearly defined observation per crop/region/time combination or another explicitly documented unit of analysis.
- Licence: usage and redistribution conditions reviewed before publishing data or a trained artifact.

4.3 Candidate Data Dictionary (To Be Confirmed)

Field / concept	Role	Expected type	Validation requirement
Crop / Item	Feature	Categorical string	Normalize whitespace/case; reject unknown values unless supported.
Yield	Target (y)	Numeric	Confirm unit and meaning; finite positive or plausible value per source.
Average rainfall	Feature	Numeric	Confirm unit (paper discusses mm); non-negative and plausible.
Average temperature	Feature	Numeric	Confirm unit (paper discusses °C); check impossible values.
Pesticide use	Feature, if present	Numeric	Confirm source field, unit, year/geography alignment, and missingness.
Year	Metadata / possible feature	Integer	Preserve for auditing; use time-aware validation if forecasting future years.
Region / country	Metadata or feature	Categorical	Confirm geographic scope and whether deployment inputs can supply it.

4.4 Data Processing Requirements

- Preserve a read-only copy of the raw data; never overwrite the source file.
- Check file existence, parse errors, empty data, expected columns, data types, and duplicate rows.
- Report missing values per column and missingness in the target separately.
- Document each removal, imputation, type conversion, and outlier decision with counts and rationale.
- Do not automatically delete outliers solely because they are statistically unusual; investigate whether they are errors or valid extremes.
- Avoid target leakage and duplicate/group leakage across train and test sets.
- Keep all learned preprocessing (imputation, encoding, scaling where needed) inside a scikit-learn Pipeline/ColumnTransformer and fit it on training folds only.
- Record the final row count after each processing step so that results can be audited.

4.5 Target Unit and Provenance Checks

The target unit must be consistent across training data, metrics, API responses, and UI labels. The research paper discusses yield in hg/ha but also reports error in t/ha in its results. Since 1 tonne/ha equals 10 hectograms/ha, the implementation must resolve and document the conversion before interpreting metric magnitudes. No metric should be presented without its target unit.

4.6 Dataset Split Strategy

- Default reproduction experiment: fixed, documented 80/20 train/test split only after confirming that random splitting is appropriate.
- Tune models using cross-validation on the training portion; do not use the test set for model selection.
- If year is meaningful and future prediction is claimed, include a chronological holdout or time-based validation.
- If observations share region, farm, or repeated entities, consider grouped splitting to prevent near-duplicate leakage.
- Publish both the split method and its limitations with the final metrics.

5. External Interface Requirements

5.1 User Interface

- The home screen shall identify the project and state that predictions are estimates.
- The interface shall provide fields only for validated model features, with clear labels and units.

- Categorical controls shall show supported values; numeric controls shall enforce valid numeric input and documented ranges.
- The result screen shall display the estimated yield, unit, selected crop/input values, and a concise limitations notice.
- The interface shall display actionable validation errors rather than a Python traceback.
- If a user input is outside the observed training range, the application shall show a warning or reject the input according to the documented policy.

5.2 Software Interfaces

Interface	Requirement
Python modules	Loader, validation, training, and inference functions shall have documented inputs and outputs.
Saved model	The complete fitted preprocessing-plus-estimator pipeline shall be loaded as one artifact where practical.
Streamlit	Optional primary demonstration UI; must call the shared inference function.
FastAPI	Optional REST interface; if implemented, provide a documented request/response schema and validation errors.
File system	Use configurable paths; do not rely on the current working directory implicitly.

5.3 Optional API Contract

If FastAPI is included, the service may expose **GET /health** and **POST /predict**. The final field names must match the validated dataset/model schema.

Endpoint	Input	Successful response	Failure behavior
GET /health	No body	Service status and model-loaded status	Non-success status if required model is unavailable
POST /predict	JSON object with supported feature fields	Numeric prediction, target unit, model/version identifier, warnings list	HTTP 4xx for invalid inputs; 5xx only for unexpected server failures

5.4 Hardware Interfaces

No specialized hardware interface is required for the initial release. A typical laptop capable of running Python, pandas, and scikit-learn is sufficient for the tabular dataset size described by the reference paper. Actual resource needs must be checked against the selected dataset and model.

6. Functional Requirements

ID	Requirement	Specification	Verification
FR-01	Dataset loading	The system shall load the configured dataset from a supported file path and return a structured table or a clear error.	Unit test with valid and missing file
FR-02	Schema validation	The system shall check required columns, types, empty files, and target availability before analysis/training.	Tests for missing column, wrong type, empty file
FR-03	Data-quality report	The system shall report row/column counts, missing values, duplicate rows, and invalid numeric values.	Report counts match independently checked sample
FR-04	Reproducible cleaning	The system shall apply documented transformations without overwriting raw data.	Run twice with same inputs and compare outputs
FR-05	EDA outputs	The system shall generate summary statistics and charts for target distribution, crop counts, feature distributions, and relationships where valid.	Expected reports/figures are created
FR-06	Training	The system shall train at least one baseline regressor and one stronger candidate using a reproducible split.	Training completes and records split/seed

ID	Requirement	Specification	Verification
FR-07	Model comparison	The system shall compare candidates using MAE, RMSE, and R^2 on consistent validation data.	Metrics computed with consistent target units
FR-08	Tuning	The system shall optionally tune model hyperparameters using training data only.	Test confirms test set is not passed to tuning
FR-09	Model persistence	The system shall save the selected fitted pipeline and metadata including feature order, target unit, and version.	Artifact can be reloaded and predicts on a test input
FR-10	Prediction	The system shall validate a single input record and return a numeric prediction in the documented unit.	Valid/invalid input tests
FR-11	Prediction explanation	The UI shall show the prediction, units, input summary, and caveat; model interpretation may be included when supported.	Manual UI review
FR-12	Error handling	The system shall provide clear messages for invalid inputs, missing model files, unsupported categories, and malformed datasets.	Negative-path tests
FR-13	Documentation	The repository shall include setup, data acquisition, training, evaluation, inference, and limitations instructions.	Fresh-environment README walkthrough
FR-14	Auditability	The system shall record dataset provenance, preprocessing decisions, model configuration, and evaluation results.	Artifacts/reports contain required metadata

7. Machine-Learning Requirements

7.1 Problem Formulation

The initial task is supervised regression: estimate a continuous crop-yield value from the supported input features. The output is not a crop-class label. The exact unit of analysis and target must be fixed after dataset audit.

7.2 Candidate Models

Model	Purpose	Notes
DummyRegressor	Sanity-check baseline	Provides a simple reference score; a complex model should outperform it.
LinearRegression	Interpretable regression baseline	Appropriate for a continuous target; inspect residuals and assumptions.
KNeighborsRegressor	Distance-based comparison	May require scaling of numeric features; sensitive to feature scale.
GradientBoostingRegressor	Primary tree-based candidate	Can model non-linear relationships; tune conservatively.
RandomForestRegressor (optional)	Additional ensemble benchmark	Useful comparison; consider training time and overfitting.

The reference paper mentions “multivariate logistic regression.” Logistic regression is generally a classification method, not a standard continuous-yield regressor. For this project's regression target, use a regression estimator such as LinearRegression unless a separate classification task is explicitly defined.

7.3 Evaluation Metrics

- MAE: average absolute error; report in the verified yield unit.
- RMSE: square-root mean squared error; report in the verified yield unit.
- R^2: comparative goodness-of-fit measure; may be negative on poor predictions and is not a guarantee of accuracy.
- Report train/validation/test strategy, data counts, random seed or date boundary, and selected model parameters.
- Include residual or error analysis where practical, not only a single headline score.

7.4 Model Selection and Reproducibility

- Select the final model based on validation results and documented criteria; use the held-out test set only for final evaluation.
- Use a fixed random seed for reproducible random splits and estimators where supported.
- Store experiment settings, package versions, metrics, and feature schema with the model artifact.
- Compare the final candidate against the DummyRegressor baseline.
- Do not claim that a high random-split R^2 proves future-year, new-region, or causal predictive performance.
- Do not promise to reproduce the paper's reported score; verify data version, target unit, split, and preprocessing first.

7.5 Interpretability and Responsible Use

- Provide feature importance or permutation importance only when the method is suitable and the interpretation is clearly qualified.
- Do not describe correlation or model feature importance as proof that a factor causes yield changes.
- Display a warning that unusual weather, soil, irrigation, seed, pest, management, and local conditions may be absent from the dataset.
- Identify data coverage gaps and avoid implying that results generalize equally across all crops, years, and regions.

8. Non-Functional Requirements

ID	Quality attribute	Requirement
NFR-01	Usability	A first-time user shall be able to enter a valid record and interpret the result without reading source code.
NFR-02	Reliability	Invalid inputs and unavailable model artifacts shall produce controlled, understandable errors.
NFR-03	Reproducibility	Documented setup and commands shall allow a fresh environment to run tests and reproduce the reported evaluation within expected numerical tolerances.
NFR-04	Maintainability	Data loading, validation, training, inference, and UI logic shall be separated into modules.
NFR-05	Performance	For the initial tabular dataset, a single local prediction should ordinarily complete within 2 seconds on a typical development laptop after the model is loaded; measure and document actual performance.
NFR-06	Portability	The project shall avoid hard-coded machine-specific absolute paths and document supported Python/library versions.
NFR-07	Security	Do not accept arbitrary uploaded executable files or deserialize untrusted model artifacts; validate file types and sizes if uploads are exposed.
NFR-08	Privacy	Do not collect personal data unless required and explicitly documented; the initial application should not need personal identifiers.
NFR-09	Traceability	Each requirement shall be linked to a test, report, or documented manual verification.
NFR-10	Transparency	Prediction units, dataset limitations, and evaluation methodology shall be visible in the documentation and user interface.

8.1 Availability and Recovery

The initial release is a local demonstration rather than a guaranteed-availability service. The application shall fail safely when required files are missing, provide an actionable message, and allow recovery by restoring the documented dataset/model artifact. Backups of source data and approved model artifacts should be maintained separately from generated temporary files.

9. Use Cases and User Workflows

UC-01: Inspect a Dataset

Item	Description
Actor	Developer / data analyst
Preconditions	Dataset file exists and its source is documented.
Main flow	Select file → load dataset → validate schema → generate data-quality summary → review EDA outputs → document cleaning decisions.
Alternative flow	If file is missing or schema is invalid, stop processing and show the specific problem; do not silently continue.
Postconditions	Data dictionary and quality report are available; raw data remains unchanged.

UC-02: Train and Evaluate a Model

Item	Description
Actor	Developer / ML engineer
Preconditions	Validated dataset and target unit are documented.
Main flow	Define features/target → create leakage-safe split → fit preprocessing and baseline → train candidates → tune on training data → evaluate once on held-out test data → save report and pipeline.
Alternative flow	If target/feature validation fails or baseline performance is implausible, stop and investigate before selecting a model.
Postconditions	Metrics, configuration, data split, and approved model artifact are recorded.

UC-03: Request a Yield Estimate

Item	Description
Actor	Demo user
Preconditions	Approved model artifact is available and model schema matches the application.
Main flow	Open interface → choose supported crop and enter feature values → submit → input validation → model inference → display estimate, unit, input summary, and caveat.
Alternative flow	If a field is missing, invalid, or unsupported, show an actionable validation message and request correction.
Postconditions	User receives an estimate or a clear error; no unsupported certainty is implied.

10. Acceptance Criteria and Verification

The initial release is accepted when the following criteria are demonstrated. Thresholds for model quality must be agreed after dataset inspection; the project must not invent an arbitrary accuracy target or claim success solely from R^2.

ID	Acceptance criterion
AC-01	A fresh setup follows the README and installs the documented dependencies.
AC-02	The loader reads the selected dataset and reports missing-file, empty-file, and schema errors clearly.
AC-03	Data-quality outputs include row count, column count, missing values, duplicates, and target-unit documentation.
AC-04	EDA produces saved charts/tables for the target and relevant crop/feature distributions.
AC-05	At least one simple baseline and one stronger regression model are trained and compared.
AC-06	Preprocessing is fitted within the training pipeline; the test set is not used for tuning.
AC-07	MAE, RMSE, and R^2 are reported on held-out data with target units and split details.
AC-08	The approved model pipeline can be loaded in a clean process and produces a finite numeric prediction for a valid input.

ID	Acceptance criterion
AC-09	Invalid input and missing model scenarios are handled without exposing a traceback in the UI.
AC-10	The UI shows estimate, unit, input summary, and limitations.
AC-11	Unit tests cover data validation and inference; an integration test exercises the main prediction path.
AC-12	README, SRS, dataset provenance, evaluation report, and limitations are present in the repository.

10.1 Testing Strategy

- Unit tests: loaders, schema checks, input validation, feature ordering, metric calculations, and inference.
- Integration tests: saved model loads and a valid sample passes through the complete prediction pipeline.
- Negative tests: missing file, missing column, invalid category, non-numeric value, NaN/infinity, missing model artifact.
- Reproducibility check: repeat the documented training/evaluation command and compare recorded configuration and results.
- Manual usability check: a first-time user completes one prediction and understands its unit and caveat.

11. Project Phases and Deliverables

Phase	Name	Key work	Deliverable
0	Environment and repository setup	Create project folders, virtual environment, dependency file, Git repository, and README skeleton.	Reproducible setup
1	Research and dataset audit	Confirm dataset source/licence, columns, units, target, row definition, and limitations.	Data source note and data dictionary
2	Ingestion and validation	Implement loader, schema checks, missing/duplicate reports, and safe paths.	Reusable loader + tests
3	Exploratory data analysis	Explore target distribution, crop frequency, feature distributions, relationships, and data quality.	Notebook and saved figures
4	Baseline definition	Define X/y, split method, metrics, DummyRegressor and LinearRegression baselines.	Baseline results
5	Preprocessing and feature engineering	Build ColumnTransformer/Pipeline for numeric/categorical features; document decisions.	Leakage-safe preprocessing pipeline
6	Model comparison	Train Gradient Boosting and optional KNN/Random Forest candidates; compare validation metrics.	Comparison table
7	Tuning and robust evaluation	Tune using training folds, run held-out test evaluation, and time-aware check if relevant.	Final metrics and error analysis
8	Interpretability and model card	Document feature influence carefully, limitations, coverage, intended use, and non-use cases.	Model card
9	Packaging and inference	Save full pipeline with metadata and implement validated prediction function.	Reusable inference module
10	User interface / optional API	Build Streamlit interface; add FastAPI only if it supports project goals.	Demonstrable application
11	Testing and portfolio packaging	Run tests, finalize README, screenshots, architecture diagram, reports, and demo.	GitHub-ready project
12	Optional advanced extension	Add time-aware prediction, external weather data, region-aware models, or explainability after baseline is stable.	Clearly scoped extension

Recommended implementation order: complete and verify each phase's deliverable before adding advanced features. Keep notebooks for exploration and move stable, reusable logic into the src package.

12. Risks, Assumptions, and Constraints

ID	Risk	Level	Mitigation
R-01	Dataset columns differ from the paper or pesticide field is absent.	High	Inspect actual file and data dictionary before freezing schema; mark unavailable features as out of scope.
R-02	Yield unit is inconsistent across sources, metrics, and UI.	High	Verify source units and conversions; include unit in every metric/prediction report.
R-03	Random split produces optimistic results due to time/entity leakage.	High	Check repeated observations and time structure; add chronological/grouped evaluation where appropriate.
R-04	Outlier removal discards valid agricultural extremes.	Medium	Investigate records and document each rule; compare results with/without justified exclusions.
R-05	High reported score is treated as proof of generalization.	High	Use baseline comparisons, held-out test evaluation, error analysis, and explicit limitations.
R-06	Raw dataset cannot be redistributed due to licence.	Medium	Document source and acquisition instructions; do not commit restricted data.
R-07	Saved model and inference schema drift apart.	Medium	Bundle preprocessing, store feature metadata, and test loading/prediction in CI or local tests.
R-08	Beginner scope expands too early into live APIs or advanced deep learning.	Medium	Finish tabular baseline and UI first; make advanced features optional.

12.1 Assumptions

- A suitable historical crop-yield dataset can be obtained legally and read locally.
- The initial release targets tabular regression, not satellite-image analysis.
- The first user interface is a local demonstration and does not require authentication.
- The exact dataset schema, feature availability, target unit, and deployment ranges remain pending validation.

12.2 Dependencies

- Python environment and compatible package versions.
- Access to the selected dataset and its documentation/licence.
- Sufficient memory/storage for the dataset and model artifacts.
- Optional Streamlit/FastAPI packages if the UI/API components are included.

13. Deployment, Maintenance, and Future Enhancements

13.1 Initial Deployment

- Run locally through documented commands in a virtual environment.
- Keep raw data, intermediate files, trained artifacts, and reports in clearly separated directories.
- Store secrets (if any are introduced later) in environment variables and exclude them from version control.
- Document how to acquire data, train a model, launch the interface, and run tests.

13.2 Model Update Procedure

- Acquire and validate a new, licensed dataset version.
- Re-run quality checks and compare schema, distributions, target units, and coverage against the previous version.
- Retrain using the documented split/evaluation strategy and compare with the current approved model.
- Promote the new model only after review of metrics, error analysis, and limitations.

- Record dataset version, code version, package versions, configuration, and evaluation results.

13.3 Future Enhancements (Not Required for Initial Release)

- Chronological forecasting and evaluation on unseen years.
- Region-specific or crop-specific models where sample sizes justify them.
- Integration with weather, soil, irrigation, satellite, or other external datasets after provenance and alignment checks.
- Explainability visualizations and uncertainty estimates where methodologically justified.
- Deployment to a managed hosting platform with monitoring and controlled model versioning.
- A more advanced API/UI and batch prediction from uploaded files.

14. References and Glossary

14.1 Primary Research Reference

Hoque et al., “Incorporating Meteorological Data and Pesticide Information to Forecast Crop Yields Using Machine Learning,” IEEE Access, 2024. DOI: 10.1109/ACCESS.2024.3383309. This reference motivates the crop-yield use case and candidate modeling approach. Reported results in the paper are not guaranteed to be reproducible on a different dataset version or evaluation protocol.

14.2 Technical References

- Python documentation: https://docs.python.org/3/
- pandas documentation: https://pandas.pydata.org/docs/
- scikit-learn documentation: https://scikit-learn.org/stable/
- Streamlit documentation: https://docs.streamlit.io/
- FastAPI documentation (optional): https://fastapi.tiangolo.com/
- The selected dataset's official source page and licence must be added to the project's README/data provenance note after the dataset is confirmed.

14.3 Glossary

Term	Meaning in this project
Inference	Using a fitted pipeline to generate a prediction for new input data.
Generalization	Performance on valid unseen data drawn from the intended use setting.
Held-out test set	Data reserved for final evaluation and not used to choose model settings.
Hyperparameter tuning	Selecting model settings using training data and validation/cross-validation.
Model card	A concise document describing a model's intended use, evaluation, limitations, and risks.
Data provenance	Record of where data came from, how it was obtained, and which version was used.

Final implementation rule: the dataset audit is the first technical gate. Do not finalize feature names, input forms, model claims, or prediction units until the real dataset has been inspected and its schema and provenance have been verified.